DOCUMENTO TÉCNICO · PRODUCCIÓN

Qué falta para poner el proyecto en producción

Gaps de seguridad de configuración, observabilidad, despliegue, cobertura de tests y continuidad operativa — verificados contra el código real, no contra lo que la documentación dice que "debería" estar.

Complementa — `arquitectura-y-frontends-nuevos.pdf` (arquitectura y estado funcional)

No repite — "qué hace cada servicio", eso ya está en el documento de arquitectura

Fecha de verificación — 18 de octubre de 2026

Resumen ejecutivo

Convención de prioridad usada en todo el documento:

5	3	1
BLOQUEANTE	IMPORTANTE	DESEABLE
NO LANZAR SIN ESTO	PRONTO DESPUÉS DE LANZAR	MEJORA, NO BLOQUEA

1. Configuración y secretos — Bloqueante
2. Exposición pública y red — Bloqueante
3. Observabilidad — Bloqueante
4. Datos y continuidad operativa — Bloqueante
5. Cobertura de tests por servicio — Importante
6. Build, CI/CD y despliegue — Bloqueante
7. Seguridad — repaso puntual — Importante
8. Frontend y app móvil — Importante
9. Deuda de arquitectura a decidir antes de crecer — Deseable

PARA LEER ESTO DE UN VISTAZO

Si sólo tenés tiempo para cuatro cosas antes de confiarle esto a usuarios reales que todavía no lo están usando: **secretos/configuración** (sección 1 — dos variables de entorno mal puestas rompen todo), **backups** (sección 4 — hoy una pérdida del servidor borra todo sin posibilidad de recuperación), **observabilidad mínima** (sección 3 — hoy nadie se entera de un error salvo que lo busque a mano) y **CI** (sección 6 — nada corre los tests automáticamente antes de que algo llegue a producción). El resto importa, pero estos cuatro son los que de verdad pueden doler.

01 🔴 BLOQUEANTE

Configuración y secretos

- `.env.example` **trae defaults inseguros que funcionan "solos" si nadie los pisa**: `DJANGO_DEBUG=True` y `DJANGO_SECRET_KEY=changeme`. Si el `.env` de producción se arma copiando el example sin revisar cada línea, el sitio queda con stack traces de Django expuestos al público y una `SECRET_KEY` conocida (compromete firmas de sesión, tokens de reset de contraseña, etc.). Antes de desplegar: `DJANGO_DEBUG=False`, `DJANGO_SECRET_KEY` generada (`get_random_secret_key()`), `DJANGO_ALLOWED_HOSTS` con el/los dominios reales (no `*`).
- `FIELD_ENCRYPTION_KEY` **vacía por default** — sin ella, `Canal.api_key` / `ServiceToken.token` / `webhook_secret` no tienen con qué cifrarse. Generar una clave Fernet real (`cryptography.fernet.Fernet.generate_key()`) y guardarla fuera del repo (gestor de secretos, no el `.env` commiteado). **Perder esta clave en producción es irrecuperable** — ningún secreto cifrado en la base se puede volver a leer sin ella; incluirla en el plan de backups (sección 4).
- `EMAIL_BACKEND` **por default es** `console` — recuperar contraseña/usuario no manda ningún email real todavía, sólo lo loguea. Configurar un proveedor SMTP real (`EMAIL_HOST` / `EMAIL_HOST_USER` / `EMAIL_HOST_PASSWORD` / `EMAIL_USE_TLS` / `DEFAULT_FROM_EMAIL`) antes de que un usuario real necesite recuperar su cuenta.
- `seed_ucd_demo` **no debe correr contra una base de producción** — crea una organización y usuarios de prueba con contraseñas conocidas/predecibles. Si se corrió alguna vez contra una base que luego pasó a producción, desactivar o eliminar esas cuentas antes del lanzamiento.
- **Revisar** `META_APP_ID` **/** `META_APP_SECRET` **/** `META_WEBHOOK_VERIFY_TOKEN` — son credenciales de la app de Meta real, no de una app de test; confirmar que están en el modo de permisos que la integración necesita (`pages_messaging`, Embedded Signup de WhatsApp) antes de ponerlas en manos de organizaciones reales.

02 BLOQUEANTE

Exposición pública y red

- **El único camino de entrada pública hoy es un Cloudflare Tunnel** (`cloudflared`, perfil `tunnel`) — y no es sólo un recurso de desarrollo/demo: hoy expone **6 hostnames reales y publicados** (`webhooks.udcchat.com.ar` → Channel Gateway, `api.udcchat.com.ar` → Core, `admin.udcchat.com.ar` → Admin Panel, `realtime.udcchat.com.ar` → Conversation Engine, `operadores.udcchat.com.ar` → Operators Desk Web, `webchat.udcchat.com.ar` → Webchat), con operadores reales atendiendo a través de `operadores.udcchat.com.ar` ahora mismo. Es, de hecho, el único borde de red del sistema — vale tratarlo con el mismo cuidado que cualquier otra arquitectura de entrada de producción, no como un atajo de desarrollo:
  - Confirmar que cada Public Hostname de Cloudflare está restringido a los paths que hace falta exponer de cada servicio (`/webhooks/*` en Channel Gateway, por ejemplo) — hoy varios reenvían el hostname completo del contenedor de destino.
  - Definir qué pasa si el túnel se cae: no hay alerta ni failover — Meta deja de poder entregar webhooks y los 6 servicios públicos (incluidos el Admin Panel y Operators Desk Web, que usan operadores reales) quedan inaccesibles hasta que alguien note el problema.
  - Evaluar si para el volumen esperado conviene además un load balancer/reverse proxy propio con TLS gestionado, en vez de depender exclusivamente de un túnel de terceros como único punto de entrada de todo el sistema.
- **CORS del Webchat Service es `*` por default** (`CORS_ORIGINS`) — correcto para una primera versión de un widget embebible en cualquier sitio, pero vale decidir explícitamente si para producción conviene restringir por organización (lista de dominios autorizados que el Admin carga al configurar el canal Webchat) en vez de aceptar cualquier origen.
- **Ningún servicio tiene rate limiting a nivel de borde** (nginx/gateway) más allá del throttling de login de Core (`ScopedRateThrottle`, sólo en `/api/auth/login/`). Un webhook de Meta mal configurado, un bot, o un pico de tráfico en el widget de Webchat no tienen ningún límite aguas arriba de la aplicación.

03 BLOQUEANTE

Observabilidad

Hoy, si algo falla en producción, el único rastro es `docker compose logs <servicio>` leído a mano:

- **Sin agregación de logs** — no hay nada que centralice logs de los 6 contenedores en un solo lugar consultable (ni siquiera algo simple tipo volcar a archivo + `docker logs` persistente). Reiniciar un contenedor pierde su historial de logs.
- **Sin tracking de errores** (Sentry o similar) — una excepción no manejada en Core hoy la captura `common.exceptions.exception_handler` (devuelve `{"error": {...}}` limpio al cliente y loguea con `logger.exception(...)`) pero ese log sólo existe en la salida de `docker logs core`, nadie se entera en el momento. Lo mismo para cualquier error no capturado en los servicios FastAPI, que ni siquiera tienen ese wrapper.
- **Sin métricas de aplicación** (latencia de endpoints, tasa de error, conexiones WebSocket activas reales) — sólo existe el healthcheck binario de Docker (`CMD curl -f .../health`), que confirma "el proceso responde" pero no "la aplicación funciona bien".
- **Sin alertas de ningún tipo** — nadie se entera si el túnel se cae, si la base se queda sin espacio, o si un canal de WhatsApp empieza a fallar silenciosamente (el chequeo de salud de canales es manual, lo dispara un Admin desde el panel).

> **MÍNIMO RAZONABLE ANTES DE PRODUCCIÓN**
>
> Un backend de logs centralizado (aunque sea simple, tipo volcar JSON estructurado a un volumen rotado), y como mínimo notificaciones por email/Slack ante un error 500 o un healthcheck caído — no hace falta una stack de observabilidad completa desde el día 1.

04 BLOQUEANTE

Datos y continuidad operativa

- **Sin backups automatizados de PostgreSQL** — `pgdata` es un volumen Docker local; si el host se pierde, se pierde todo (organizaciones, conversaciones, historial completo). Ni Core, ni ningún otro servicio, respaldan nada por su cuenta.
- `FIELD_ENCRYPTION_KEY` **y las claves JWT (** `jwt_keys` **volumen) no están en el plan de backup** — perder la clave de cifrado deja ilegibles `api_key` / `webhook_secret` / `ServiceToken.token` para siempre (no hay forma de "resetear" eso salvo pedirle a cada organización que vuelva a cargar sus credenciales de Meta). Perder las claves JWT invalida todas las sesiones activas de golpe.
- `media_files` **(avatares subidos) también vive sólo en un volumen local** — mismo riesgo, sin réplica externa.
- **Sin plan de capacity/escalado**: `docker-compose.yml` fija límites de CPU/memoria pequeños pensados para desarrollo (ej. Core 1 CPU/512MB, servicios FastAPI 0.5 CPU/256MB) — no se dimensionaron para carga real, y no hay forma de escalar horizontalmente ningún servicio FastAPI hoy: **Conversation Engine y Webchat guardan las conexiones WebSocket activas en memoria de un solo proceso** (ni siquiera `--workers 2` de uvicorn funcionaría sin romper el registro de conexiones) — limitación conocida, sigue sin resolverse.

05 IMPORTANTE

Cobertura de tests por servicio

Servicio	Archivos de test	Observación
Core	12 módulos	Suite real, corre en CI... que no existe (sección 6)
Channel Gateway	6 módulos	Cubre webhooks/envío; no cubre el camino completo con Core real (mockeado)
Conversation Engine	4 módulos	—
Webchat	**6 módulos (37 tests)**	Agregada 2026-10: salud, sesión/historial, mensaje entrante y saliente, WebSocket del visitante, `ConnectionManager`, `Service-Token`, cliente hacia Core
Operators Desk Web (React)	**0**	**En producción real (`operadores.udcchat.com.ar`) sin ningún test — ahora sí, el único gap serio que queda en esta lista**
Analytics	0	Stub — no aplica mientras siga así

El gap real y concreto más urgente es **Operators Desk Web**: es la herramienta con la que los operadores atienden conversaciones reales hoy mismo, y no tiene ni un test de componente ni un test end-to-end — cualquier regresión en "tomar"/"responder"/"derivar"/"cerrar" llega directo a producción sin red de seguridad. Webchat, que era el segundo caso más urgente, ya no lo es: tiene una suite completa (`services/webchat/tests/`) cubriendo exactamente lo que faltaba — creación de sesión, mensaje entrante válido/inválido, autenticación del mensaje saliente (`Service-Token`), y que un visitante desconectado no rompe el envío. Queda un detalle menor, no de cobertura: a diferencia de Channel Gateway, `core_client.py` todavía no tiene un cliente HTTP inyectable (sigue abriendo el suyo por método), así que esos tests parchean el import en vez de inyectar un seam real — `anti-slop-py` lo marca (`ASP003`). Portarlo al mismo patrón de `http_client.py` de Channel Gateway queda como mejora, no como bloqueante.

06 BLOQUEANTE

Build, CI/CD y despliegue

- **No existe ningún pipeline de CI** (`.github/workflows` no existe en el repo) — ningún test de ningún servicio corre automáticamente en un PR o push. Hoy la única red de seguridad es correr `manage.py test` a mano antes de mergear, lo cual depende enteramente de que la persona que mergea se acuerde de hacerlo.
- **No hay pipeline de build/deploy automatizado** — el despliegue hoy es manual (`docker compose up --build` en el servidor). No hay versionado de imágenes (todo se buildea `latest` in-place), ni un paso de rollback claro si un deploy rompe algo.
- `docker-compose.override.yml` **(hot-reload, 1 worker, bind mounts de código fuente) no debe llegar a producción** — confirmar que el despliegue real usa sólo `docker-compose.yml` sin el override (el override existe sólo para desarrollo local). Si el servidor de producción corre `docker compose up` sin más, **se está sirviendo con el reload de desarrollo activado**.
- **Gunicorn de Core corre con la configuración mínima** (ver `docker-compose.override.yml` vs. el compose base) — confirmar cuántos workers/threads tiene en el `Dockerfile` /entrypoint de producción; no quedó documentado un número pensado para carga real.

07 IMPORTANTE

Seguridad — repaso puntual

- `BUENAS-PRACTICAS-API-REST.md` documenta una auditoría propia del proyecto — repasarla antes de cerrar esta lista, probablemente tiene ítems que se solapan con esta sección pero con más detalle a nivel de endpoint.
- **Formato de error ya estandarizado** (`{"error": {"code", "detail"}}` vía `common.exceptions.exception_handler`, confirmado implementado) — ya no es un pendiente, a diferencia de lo que figuraba en una versión anterior de esta revisión.
- **Secretos cifrados en reposo ya implementado** (Fernet, sección 1) — tampoco es un pendiente; `docs/CONTEXTO_PROYECTO.md` §9 todavía dice "sigue pendiente" en un párrafo que quedó desactualizado, vale corregirlo la próxima vez que se edite esa sección para que no confunda a alguien que lo lea como estado actual.
- **Dependencias sin escaneo de vulnerabilidades automatizado** — ni `pip-audit` / `safety` para los `requirements.txt` de los 6 servicios Python, ni `npm audit` corriendo en CI para el Admin Panel (tampoco hay CI, sección 6).
- **Revisar permisos del túnel/Cloudflare y de la cuenta de Meta** usados en producción — son accesos privilegiados (pueden exponer tráfico real de clientes) que vale auditar una vez antes del lanzamiento, no sólo confiar en que "ya están configurados".

08 IMPORTANTE

Frontend y app móvil

- **Operators Desk Web existe y está operativa** (`services/operators-desk/web/` , publicada en `operadores.udcchat.com.ar`) — corrección respecto a una versión anterior de este documento, que la daba por no construida. Dos pendientes reales, sí: (1) **no está en** `docker-compose.yml` — no tiene su propio service block orquestado junto al resto, así que un `docker compose up` desde cero no la levanta ni la reconstruye; se construye/despliega aparte, un punto ciego para cualquiera que no sepa que ese paso extra existe. (2) **cero tests** (sección 5) para la herramienta que los operadores usan en producción hoy mismo.
- **App Android sin refresh token** — `Sesion` sólo persiste el `access` token (vence a los 30 minutos); al vencer, la app fuerza logout en vez de usar `POST /api/auth/refresh/` . Para uso real de un operador en un turno largo, esto es una fricción diaria, no un detalle menor.
- **App Android sin notificaciones push** — explícitamente fuera de alcance sin pedido explícito, pero vale decidir conscientemente si eso es aceptable para operadores de producción (hoy dependen de polling + WebSocket con la app abierta).
- **App Android no tiene firma/distribución de producción documentada acá** — hay un `keystore/` y un `apk/` en el repo; confirmar que el proceso de firmado y distribución (Play Store interno, APK directo, etc.) está decidido antes de repartir la app a operadores reales.

09 DESEABLE

Deuda de arquitectura a decidir antes de crecer

- ~~**"Operators Desk" (FastAPI, puerto 8004) es un stub sin ningún consumidor**~~ — **resuelto (2026-10)**: se dio de baja del repo y del `docker-compose.yml` junto con su `ServiceToken`. Operators Desk Web (la app React, real y en producción, sección 8) nunca le habló a este servicio — siempre fue directo a Core.
- `conversations.asignacion.asignar_round_robin` **sigue en el código sin ningún call site** — no es un problema de producción en sí, pero si nunca se va a usar, es candidato a eliminarse (o a quedar documentado como "utilidad a propósito, para una asignación explícita futura") para que no se lea como código muerto accidental.
- **Analytics no tiene fecha ni sigue** — el rol Data Manager no tiene ninguna pantalla hoy. Si hay un compromiso comercial de reportes, vale tenerlo en el radar de planificación aunque no sea bloqueante técnico para lanzar el resto.